



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ
на программный модуль
«Finlytics: Офлайн-менеджер финансов»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Руководитель проектной работы

Мельников Д.А.

(подпись руководителя)

Москва 2026

Оглавление

1. Объект испытаний	4
1.1. Наименование программного продукта	4
1.2. Назначение и область применения	4
1.3. Краткая характеристика программного продукта	4
1.3.1. Основные функциональные возможности	4
1.3.2. Технические характеристики	5
1.3.3. Архитектура приложения	5
1.3.4. Состав программного продукта	5
2. Цель испытаний	6
2.1. Основные цели проведения испытаний	6
2.2. Критерии оценки результатов испытаний	6
3. Требования к программе	7
3.1. Общие положения	7
3.2. Функциональные требования	7
3.2.1. Управление операциями	7
3.2.2. Управление категориями	7
3.2.3. Аналитика и просмотр данных	8
3.3. Нефункциональные требования	8
3.4. Требования к документации	9
4. Требования к программной документации	10
4.1. Состав программной документации, предъявляемой на испытания	10
4.2. Требования к содержанию программной документации	10
4.2.1. Техническое задание	10
4.2.2. Отчет по проекту	10
4.2.3. Программа и методика испытаний	11
4.2.4. Исходный код программы	11
4.2.5. Руководство пользователя	11
4.2.6. Протокол испытаний	11
4.3. Порядок предоставления документации на испытания	12
4.4. Особые требования к документации	12
4.4.1. Требования к документации разработчика (KDoc)	12
4.4.2. Требования к документации пользователя (GitHub Wiki)	12
5. Состав и порядок проведения испытаний	13
5.1. Состав испытаний	13
5.1.1. Предварительные (автономные) испытания	13
5.1.2. Комплексные (системные) испытания	13
5.1.3. Приемочные испытания	14
5.2. Средства испытаний	14
5.2.1. Технические средства	14
5.2.2. Программные средства	15
5.3. Порядок проведения испытаний	16
5.3.1. Общий порядок проведения испытаний	16
5.3.2. Порядок проведения предварительных (автономных) испытаний	16
5.3.3. Порядок проведения комплексных (системных) испытаний	16
5.3.4. Порядок проведения приемочных испытаний	17

5.3.5.	Условия досрочного прекращения испытаний	18
5.4.	Требования к персоналу, проводящему испытания	18
6.	Методы испытаний	19
6.1.	Общие положения	19
6.2.	Методы испытаний функциональных требований	19
6.2.1.	Метод проверки требования ФТ-01 (добавление операции)	19
6.2.2.	Метод проверки требования ФТ-02 (редактирование операции).....	19
6.2.3.	Метод проверки требования ФТ-03 (удаление операции)	20
6.2.4.	Метод проверки требования ФТ-04 (предустановленные категории)	20
6.2.5.	Метод проверки требования ФТ-05 (создание пользовательских категорий) 21	
6.2.6.	Метод проверки требования ФТ-06 (удаление категорий с проверкой использования).....	21
6.2.7.	Метод проверки требования ФТ-07 (финансовая сводка)	22
6.2.8.	Метод проверки требования ФТ-08 (визуализация данных)	22
6.2.9.	Метод проверки требования ФТ-09 (фильтрация истории).....	23
6.2.10.	Метод проверки требования ФТ-10 (прогресс-бар «Гонка с призраком»).....	23
6.3.	Методы испытаний нефункциональных требований.....	24
6.3.1.	Метод проверки требования НФТ-01 (производительность фильтрации).....	24
6.3.2.	Метод проверки требования НФТ-02 (время запуска приложения)	24
6.3.3.	Метод проверки требования НФТ-03 (безопасность данных от SQL- инъекций)	24
6.3.4.	Метод проверки требования НФТ-04 (кроссплатформенность)	25
6.3.5.	Метод проверки требования НФТ-05 (целостность данных)	26
6.3.6.	Метод проверки требования НФТ-06 (удобство использования)	26
6.4.	Методы испытаний требований к документации.....	27
6.4.1.	Метод проверки требования ДТ-01 (документация разработчика)	27
6.4.2.	Метод проверки требования ДТ-02 (документация пользователя).....	27

1. Объект испытаний

1.1. Наименование программного продукта

Полное наименование: Программный продукт «Finlytics: Офлайн-менеджер финансов».

Краткое наименование: Finlytics или Система.

Обозначение версии: 1.0.0

1.2. Назначение и область применения

Программа предназначена для автоматизации процесса учета личных финансов физическими лицами, включая:

- Фиксацию доходов и расходов с детализацией по категориям;
- Ведение истории финансовых операций;
- Анализ структуры расходов и доходов с помощью визуальных диаграмм;
- Контроль текущего баланса и динамики трат;
- Управление пользовательскими категориями операций.

Основная область применения — личное финансовое планирование и бюджетирование для физических лиц.

1.3. Краткая характеристика программного продукта

1.3.1. Основные функциональные возможности

Программа предоставляет пользователю три основных блока функциональности.

Первый блок — управление финансовыми операциями. Пользователь может добавлять новые операции (как доходы, так и расходы), указывая сумму, категорию, дату и необязательное описание. Любую существующую операцию можно отредактировать или удалить. Система автоматически пересчитывает статистику при любых изменениях.

Второй блок — управление категориями. При первом запуске приложение автоматически создает предустановленный набор категорий для доходов (например, «Зарплата», «Стипендия») и расходов (например, «Еда», «Транспорт», «Жилье»). Пользователь может создавать собственные категории, а также удалять их. Важно, что система блокирует удаление категории, если она уже используется хотя бы в одной операции — это защищает целостность данных.

Третий блок — аналитика и визуализация. На главном экране отображается финансовая сводка: общая сумма доходов, расходов и текущий баланс за выбранный период. Пользователь может фильтровать данные по временным промежуткам (день, неделя, месяц, год или все время). Распределение доходов или расходов по категориям наглядно представляется в виде круговой диаграммы. Кроме того, реализован прогресс-бар «Гонка с призраком», который показывает соотношение расходов текущего периода с аналогичным предыдущим периодом (например, текущий месяц в сравнении с прошлым месяцем). В разделе «История» доступен полный список всех операций с возможностью

фильтрации как по типу (все, только доходы или только расходы), так и по временному периоду.

1.3.2. Технические характеристики

Таблица 1 - Технические характеристики

Параметр	Значение
Язык реализации	Kotlin (версия 1.9.24)
UI-фреймворк	Jetpack Compose for Desktop (версия 1.6.11)
Система хранения данных	SQLite (встраиваемая СУБД)
Система сборки	Gradle (Kotlin DSL)
Формат распространения	Исполняемый JAR-файл
Требования к окружению	Java Runtime Environment (JRE) версии 21 или выше

1.3.3. Архитектура приложения

Приложение построено на основе чистой многослойной архитектуры с реализацией паттерна MVVM (Model-View-ViewModel):

Таблица 2 - Архитектура приложения

Слой	Назначение	Ключевые компоненты
Слой представления (View)	Отображение пользовательского интерфейса и обработка действий пользователя	OverviewScreen, HistoryScreen, SettingsScreen, NavigationBar, PieChart, OperationDialog
Слой логики представления (ViewModel)	Управление состоянием UI, бизнес-логика, координация данных	FinanceViewModel, FinanceState
Слой данных (Repository)	Абстракция источников данных, единая точка доступа к данным	FinanceRepository
Слой хранения (Data Source)	Непосредственная работа с БД, CRUD-операции	DatabaseConfig, DatabaseInitializer, AddCategory, GetIncomeTransactions и др.

1.3.4. Состав программного продукта

Программный продукт включает в себя исполняемый файл Finlytics.jar, который является точкой входа в приложение. При первом запуске автоматически создается файл базы данных Finlytics.db, где хранятся все операции и категории пользователя. Дополнительных конфигурационных файлов не требуется — все настройки используют значения по умолчанию, а отладочная информация выводится в консоль (что удобно при запуске из командной строки).

2. Цель испытаний

2.1. Основные цели проведения испытаний

Испытания программного продукта «Finlytics: Офлайн-менеджер финансов» проводятся с целью подтверждения соответствия разработанного приложения требованиям, установленным в Техническом задании и Матрице требований, а также для оценки готовности продукта к опытной эксплуатации и последующему внедрению.

В ходе испытаний решаются следующие основные задачи:

- Проверка полноты реализации функциональных требований;
- Проверка соблюдения нефункциональных требований;
- Оценка удобства пользовательского интерфейса;
- Выявление дефектов и недостатков;
- Подготовка заключения о готовности продукта.

2.2. Критерии оценки результатов испытаний

Оценка результатов испытаний производится по следующим категориям:

Таблица 3 - Категории оценки результатов

Категория	Критерий	Решение
Принято	Все функциональные и нефункциональные требования выполнены в полном объеме. Дефекты отсутствуют, либо выявлены незначительные дефекты (косметические, не влияющие на работоспособность), которые не препятствуют эксплуатации.	Продукт готов к внедрению.
Принято условно	Выявлены дефекты среднего приоритета, не блокирующие выполнение основных функций (например, редкие сбои при определенных условиях, незначительное несоответствие интерфейса макетам).	Продукт может быть передан в опытную эксплуатацию с обязательством устранения замечаний в установленный срок (до 5 рабочих дней).
Не принято	Выявлены критические дефекты, блокирующие выполнение основных функций (невозможность добавления операции, потеря данных, невозможность запуска приложения). Либо выявлено несоответствие более чем 20% функциональных требований.	Продукт требует доработки. Назначается повторный цикл испытаний после устранения критических замечаний.

3. Требования к программе

3.1. Общие положения

В настоящем разделе приведен перечень требований к программному продукту «Finlytics: Офлайн-менеджер финансов», которые подлежат проверке в ходе испытаний. Все требования сформулированы на основе Технического задания и Матрицы требований, разработанных на этапе планирования проекта.

Для каждого требования указан его уникальный идентификатор, содержательное описание и приоритет проверки. Демонстрации заказчику подлежат все требования, перечисленные ниже, за исключением требований с низким приоритетом, которые могут быть проверены в рабочем порядке (без специальной демонстрации).

3.2. Функциональные требования

3.2.1. Управление операциями

Требование ФТ-01 (добавление операции). Программа должна предоставлять пользователю возможность добавлять новые финансовые операции. При добавлении операции пользователь должен указать тип операции (доход или расход), сумму, категорию и дату. Описание операции является необязательным полем. Приоритет проверки — высокий.

Требование ФТ-02 (редактирование операции). Программа должна предоставлять пользователю возможность редактировать существующие финансовые операции. При редактировании пользователь должен иметь возможность изменить любой параметр операции: тип, сумму, категорию, дату или описание. Приоритет проверки — высокий.

Требование ФТ-03 (удаление операции). Программа должна предоставлять пользователю возможность удалять существующие финансовые операции. При удалении операции она должна исчезать из списка истории и переставать учитываться в статистике (баланс, доходы, расходы, диаграммы). Приоритет проверки — высокий.

3.2.2. Управление категориями

Требование ФТ-04 (предустановленные категории). При первом запуске приложения программа должна автоматически создавать предустановленный набор категорий для доходов и расходов. Для доходов должны быть созданы категории: «Зарплата», «Стипендия», «Фриланс», «Инвестиции», «Подарок». Для расходов должны быть созданы категории: «Еда», «Транспорт», «Жилье», «Развлечения», «Образование», «Здоровье», «Одежда», «Коммунальные услуги». Приоритет проверки — высокий.

Требование ФТ-05 (создание пользовательских категорий). Программа должна предоставлять пользователю возможность создавать новые категории как для доходов, так и для расходов. Созданная категория должна появляться в соответствующем списке и становиться доступной для выбора при добавлении или редактировании операций. Приоритет проверки — высокий.

Требование ФТ-06 (удаление категорий с проверкой использования). Программа должна предоставлять пользователю возможность удалять существующие категории. При

этом система должна блокировать удаление категории, если она используется хотя бы в одной финансовой операции. При попытке удаления такой категории программа должна отображать понятное пользователю предупреждение. Приоритет проверки — высокий.

3.2.3. Аналитика и просмотр данных

Требование ФТ-07 (финансовая сводка). На главном экране программы должна отображаться финансовая сводка: общая сумма доходов за выбранный период, общая сумма расходов за выбранный период и текущий баланс (разница между доходами и расходами). Пользователь должен иметь возможность выбирать временной период для отображения статистики (день, неделя, месяц, год, всё время). Приоритет проверки — высокий.

Требование ФТ-08 (визуализация данных). На главном экране программы должна отображаться круговая диаграмма, показывающая распределение доходов или расходов по категориям за выбранный период. Пользователь должен иметь возможность переключаться между режимами отображения «Доходы» и «Расходы». При переключении диаграмма и список категорий должны обновляться соответствующим образом. Приоритет проверки — высокий.

Требование ФТ-09 (фильтрация истории). На экране истории операций программа должна предоставлять пользователю возможность фильтровать список операций по двум критериям: по типу операции (все операции, только доходы или только расходы) и по временному периоду (день, неделя, месяц, год, всё время). При изменении параметров фильтрации список операций должен обновляться мгновенно. Приоритет проверки — высокий.

Требование ФТ-10 (прогресс-бар «Гонка с призраком»). При выборе временного периода, отличного от «Всё время», на главном экране должен отображаться прогресс-бар «Гонка с призраком», показывающий соотношение расходов текущего периода с расходами аналогичного предыдущего периода (например, текущий месяц в сравнении с прошлым месяцем). Прогресс-бар должен иметь цветовую индикацию: зеленый при снижении расходов (менее 80% от предыдущего периода), желтый при незначительном изменении (от 80% до 100%), красный при превышении расходов (более 100%). Приоритет проверки — средний.

3.3. Нефункциональные требования

Требование НФТ-01 (производительность фильтрации). Время отклика интерфейса на действия пользователя (открытие экрана, применение фильтра, переключение периода) не должно превышать 500 миллисекунд при условии наличия в базе данных до 1000 операций. Приоритет проверки — высокий.

Требование НФТ-02 (время запуска приложения). Время от момента запуска исполняемого JAR-файла до полной загрузки главного окна приложения не должно превышать 3 секунд при условии наличия в базе данных до 1000 операций. Приоритет проверки — средний.

Требование НФТ-03 (безопасность данных). Программа должна использовать параметризованные SQL-запросы (PreparedStatement) для всех операций взаимодействия с

базой данных, что исключает возможность внедрения SQL-инъекций через пользовательский ввод. Приоритет проверки — высокий.

Требование НФТ-04 (кроссплатформенность). Программа должна корректно работать на операционных системах Windows 10 и 11, macOS 11 и выше, а также на Linux-дистрибутивах с поддержкой JRE 21 (включая Ubuntu 20.04 и новее, Fedora 35 и новее, Debian 11 и новее). На всех целевых платформах интерфейс должен отображаться корректно, а все функции — выполняться без ошибок. Приоритет проверки — средний.

Требование НФТ-05 (целостность данных). При некорректном завершении работы приложения (закрытие окна во время выполнения операции, выключение компьютера) база данных не должна повреждаться. Все успешно сохраненные до момента сбоя операции должны оставаться в базе данных. Приоритет проверки — средний.

Требование НФТ-06 (удобство использования). Все ключевые элементы управления (кнопки, поля ввода, переключатели) должны быть снабжены всплывающими подсказками, поясняющими их назначение. При вводе некорректных данных (отрицательная сумма, неверный формат даты) программа должна отображать понятные пользователю сообщения об ошибке. Приоритет проверки — низкий.

3.4. Требования к документации

Требование ДТ-01 (документация разработчика). Исходный код программы должен быть документирован в соответствии со стандартом KDoc для языка Kotlin. Все публичные классы, функции и свойства должны содержать комментарии с описанием их назначения, параметров и возвращаемых значений. Приоритет проверки — низкий.

Требование ДТ-02 (документация пользователя). Программа должна сопровождаться документацией пользователя, содержащей описание всех функций приложения, инструкции по установке и запуску, а также ответы на часто возникающие вопросы. Документация должна быть оформлена в виде GitHub Wiki. Приоритет проверки — низкий.

4. Требования к программной документации

4.1. Состав программной документации, предъявляемой на испытания

Для проведения испытаний программного продукта «Finlytics: Офлайн-менеджер финансов» предоставляется следующий комплект программной документации:

Таблица 4 - Состав программной документации

№ п/п	Наименование документа	Формат предоставления
1	Техническое задание на создание автоматизированной системы	Электронный (PDF, DOCX)
2	Отчет по проекту (сводный документ, содержащий описание архитектуры, диаграммы, результаты тестирования)	Электронный (PDF, DOCX)
3	Программа и методика испытаний	Электронный (PDF, DOCX)
4	Исходный код программы	Репозиторий GitHub
5	Руководство пользователя	GitHub Wiki
6	Протокол испытаний (формируется по результатам)	Электронный (PDF, DOCX)

4.2. Требования к содержанию программной документации

4.2.1. Техническое задание

Техническое задание должно быть разработано в соответствии с требованиями ГОСТ 34.602-2020.

Документ должен содержать следующие разделы: общие сведения; назначение и цели создания системы; характеристика объекта автоматизации; требования к системе в целом; требования к функциям (задачам), выполняемым системой; требования к видам обеспечения; состав и содержание работ по созданию системы; порядок контроля и приемки системы; требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие; требования к документированию.

4.2.2. Отчет по проекту

Отчет по проекту должен содержать описание всех этапов разработки программного продукта.

В состав отчета входят: анализ целевой аудитории и требований к системе; матрица требований; техническое задание; план реализации проекта; поведенческие и структурные диаграммы (Use Case, последовательности, классов, объектов); модель базы данных; архитектура приложения с обоснованием выбора технологий; проектирование пользовательского интерфейса с макетами экранов; листинги ключевых компонентов

фронтенда и бэкенда; документация разработчика (KDoc) и пользователя (GitHub Wiki); результаты тестирования (тестовые сценарии, чек-лист, дополненная матрица требований).

4.2.3. Программа и методика испытаний

Настоящий документ должен быть разработан в соответствии с требованиями ГОСТ 19.301-79 и содержать следующие разделы: объект испытаний; цель испытаний; требования к программе; требования к программной документации; средства и порядок испытаний; методы испытаний.

4.2.4. Исходный код программы

Исходный код должен быть размещен в публичном репозитории на платформе GitHub по адресу: <https://github.com/Hohrandrey/Finlytics>.

Репозиторий должен содержать полную структуру проекта, включая все модули фронтенда и бэкенда, файлы ресурсов, настройки системы сборки Gradle.

Доступ к репозиторию должен быть открыт для чтения. Вся команда разработки (Враженко Д.О., Павлов Н.С., Хохряков А.Ю.) должна иметь права на запись в репозиторий.

4.2.5. Руководство пользователя

Руководство пользователя должно быть оформлено в виде GitHub Wiki по адресу: <https://github.com/Hohrandrey/Finlytics/wiki>.

Документ должен содержать следующие разделы: введение (назначение и область применения программы); требования к аппаратному и программному обеспечению; инструкция по установке и запуску (включая требования к JRE); описание пользовательского интерфейса (главный экран, экран истории, экран настроек); инструкции по выполнению основных операций (добавление, редактирование, удаление операций; управление категориями; фильтрация данных); описание функционала «Гонка с призракoм»; ответы на часто задаваемые вопросы; информация о версиях и контакты разработчиков.

4.2.6. Протокол испытаний

Протокол испытаний формируется по результатам проведения испытаний и содержит: ссылку на настоящую Программу и методику испытаний; перечень проведенных проверок с указанием дат и времени; количество выявленных дефектов с классификацией по степени критичности (критические, значительные, незначительные); ссылки на соответствующие тестовые сценарии (ТС-01 — ТС-10); итоговое заключение о готовности продукта к внедрению с указанием категории («Принято», «Принято условно», «Не принято»); подписи лиц, проводивших испытания.

4.3. Порядок предоставления документации на испытания

Таблица 5 - Порядок предоставления документации на испытания

Этап	Документы	Срок предоставления
До начала испытаний	Техническое задание, Отчет по проекту, Программа и методика испытаний, Исходный код, Руководство пользователя	Не позднее чем за 3 рабочих дня до начала испытаний
В ходе испытаний	Протокол испытаний (предварительная версия с выявленными дефектами)	По завершении каждого этапа испытаний
По окончании испытаний	Протокол испытаний (итоговая версия), Акт приемки (при успешном завершении)	В день завершения приемочных испытаний

4.4. Особые требования к документации

4.4.1. Требования к документации разработчика (KDoc)

Документация разработчика формируется автоматически с использованием инструмента Dokka, встроенного в систему сборки Gradle.

Команда для генерации документации: `./gradlew dokkaHtml`. Сгенерированная документация должна быть доступна в формате HTML.

При проверке документации контролируется наличие KDoc-комментариев для всех публичных классов, функций и свойств. Комментарии должны содержать описание назначения элемента, а также теги `@param` (для параметров функции), `@return` (для возвращаемого значения) и `@throws` (для выбрасываемых исключений) при необходимости.

4.4.2. Требования к документации пользователя (GitHub Wiki)

Документация пользователя должна быть доступна в виде Wiki-страниц на платформе GitHub. Допускается использование форматирования Markdown.

Все страницы должны быть связаны между собой навигационными ссылками. Документация должна быть актуальной на момент проведения испытаний и соответствовать версии программного продукта, предъявляемой к приемке.

5. Состав и порядок проведения испытаний

5.1. Состав испытаний

Испытания программного продукта «Finlytics: Офлайн-менеджер финансов» включают в себя три последовательных этапа: предварительные (автономные) испытания, комплексные (системные) испытания и приемочные испытания. Каждый этап имеет собственный состав проверок и критерии перехода к следующему этапу.

5.1.1. Предварительные (автономные) испытания

Предварительные испытания проводятся с целью проверки работоспособности каждого модуля приложения в отдельности, до их интеграции в единую систему. В состав предварительных испытаний входят:

- Модульное тестирование backend-слоя: проверка методов репозитория (FinanceRepository), проверка CRUD-операций с категориями (AddCategory, DeleteCategory, GetIncomeCategories, GetExpensesCategories), проверка CRUD-операций с транзакциями доходов и расходов (AddIncomeTransaction, UpdateIncomeTransaction, DeleteIncomeTransaction и аналогичные для расходов).
- Автономное тестирование пользовательского интерфейса: проверка каждого экрана приложения в изоляции (OverviewScreen, HistoryScreen, SettingsScreen), проверка каждого диалогового компонента (OperationDialog, AddCategoryDialog, диалогов подтверждения удаления), проверка кастомных компонентов (PieChart, GhostRaceProgressBar), проверка навигации между экранами.
- Проверка валидаторов: проверка корректности проверки вводимых данных (сумма должна быть положительным числом, дата должна соответствовать формату и не превышать допустимый диапазон, категория должна быть выбрана из списка).

5.1.2. Комплексные (системные) испытания

Комплексные испытания проводятся с целью проверки взаимодействия всех компонентов системы в целом, а также проверки соответствия приложения функциональным и нефункциональным требованиям. В состав комплексных испытаний входят:

- Сквозное функциональное тестирование: полный цикл работы с операцией (добавление — редактирование — удаление); работа с категориями (создание — использование в операции — попытка удаления); фильтрация данных на главном экране и в истории; переключение между режимами «Доходы» и «Расходы»; работа прогресс-бара «Гонка с призраком».
- Тестирование валидации и обработки ошибок: проверка реакции системы на ввод отрицательной суммы, ввод нулевой суммы, ввод некорректной даты,

попытку сохранения операции без выбора категории, попытку удаления используемой категории.

- Тестирование производительности: проверка времени отклика интерфейса при фильтрации данных, проверка времени запуска приложения.
- Тестирование безопасности: проверка защищенности от SQL-инъекций (статический анализ кода и динамическое тестирование).
- Тестирование совместимости: проверка работоспособности приложения на различных операционных системах (Windows, Linux, macOS).
- Тестирование надежности: проверка сохранения целостности базы данных при некорректном завершении работы приложения.

5.1.3. Приемочные испытания

Приемочные испытания проводятся с целью подтверждения соответствия программного продукта требованиям Технического задания и готовности к внедрению. Приемочные испытания проводятся в присутствии заказчика (преподавателя). В состав приемочных испытаний входят:

- Демонстрация всех ключевых функций приложения: запуск и инициализация базы данных; добавление, редактирование и удаление операций; просмотр истории с фильтрацией; просмотр главного экрана с переключением между доходами и расходами; фильтрация данных по периодам; отображение круговой диаграммы; работа прогресс-бара «Гонка с призраком»; управление категориями.
- Демонстрация обработки ошибок: показ сообщений об ошибках при вводе некорректных данных.
- Сверка реализованного функционала с Техническим заданием.
- Оформление акта приемки или заключения о доработке.

5.2. Средства испытаний

5.2.1. Технические средства

Для проведения испытаний используется следующее аппаратное обеспечение.

Основной тестовый стенд (для функционального тестирования) представляет собой персональный компьютер с процессором Intel Core i5-12400 (6 ядер, 2.5 ГГц), оперативной памятью 16 ГБ и твердотельным накопителем SSD объемом 512 ГБ. Данный стенд является основным для проведения функционального и регрессионного тестирования.

Стенд для тестирования производительности представляет собой персональный компьютер с процессором Intel Core i3-10100 (4 ядра, 3.6 ГГц), оперативной памятью 8 ГБ и твердотельным накопителем SSD объемом 256 ГБ. Данный стенд приближен к минимальным требованиям и предназначен для проверки времени отклика и времени запуска приложения.

Стенды для тестирования кроссплатформенности включают три устройства: компьютер с операционной системой Windows 11 Pro (22H2); компьютер с операционной системой Ubuntu 22.04 LTS; ноутбук Apple MacBook Air с процессором M1 и операционной

системой macOS 13 (Ventura) — при наличии доступа. При отсутствии физического доступа к macOS допускается использование удаленного доступа к виртуальной машине.

Дополнительное оборудование включает электронный секундомер (или встроенный секундомер мобильного телефона) для проведения замеров времени, а также средства видеозаписи экрана для последующего покадрового анализа (при необходимости).

5.2.2. Программные средства

Для проведения испытаний используется следующее программное обеспечение.

Среда выполнения. На всех тестовых стендах должна быть установлена Java Runtime Environment (JRE) версии 21 или выше. Для проверки совместимости с различными версиями JRE допускается проведение дополнительных проверок с JRE версий 17, 23 и 25.

Средства модульного тестирования. Для автоматизированного тестирования backend-логики используется фреймворк JUnit 5. Модульные тесты запускаются из среды разработки IntelliJ IDEA или из командной строки с помощью Gradle (команда `./gradlew test`). Результаты выполнения модульных тестов сохраняются в формате HTML в директории `build/reports/tests/test`.

Средства тестирования пользовательского интерфейса. Для автоматизированного тестирования UI используется встроенный фреймворк Compose UI Test, который позволяет эмулировать действия пользователя (клики, ввод текста) и проверять состояние UI-компонентов. UI-тесты запускаются аналогично модульным тестам через Gradle.

Средства нагрузочного тестирования. Для проверки времени отклика при большом количестве операций используется инструмент Apache JMeter версии 5.6 или выше. JMeter позволяет эмулировать последовательные запросы к базе данных и замерять время выполнения операций фильтрации. Для подготовки тестовых данных (генерация 1000 случайных операций) используется специальный скрипт на языке Kotlin, запускаемый отдельно перед началом нагрузочных испытаний.

Средства тестирования безопасности. Для проверки защищенности от SQL-инъекций используется статический анализ кода (визуальная проверка наличия `PreparedStatement` во всех классах, работающих с БД) и динамическое тестирование — ручной ввод SQL-кода в поля ввода приложения.

Средства просмотра базы данных. Для контроля содержимого базы данных в процессе испытаний используется SQLite Browser (DB Browser for SQLite) версии 3.12 или выше. С его помощью проверяется корректность записи, обновления и удаления данных в таблицах `Income_categories`, `expenses_categories`, `Income_transactions` и `expenses_transactions`.

Средства логирования. Приложение выводит отладочную информацию в консоль. Для просмотра логов при запуске из командной строки используется стандартный вывод терминала. При запуске из среды разработки IntelliJ IDEA логи отображаются во встроенной консоли.

Средства управления версиями и документацией. Исходный код хранится в репозитории GitHub. Для просмотра кода и истории изменений используется веб-интерфейс GitHub или клиентское приложение Git. Документация пользователя размещена в GitHub Wiki и доступна через веб-браузер.

5.3. Порядок проведения испытаний

5.3.1. Общий порядок проведения испытаний

Испытания проводятся в строгой последовательности: сначала предварительные (автономные) испытания, затем комплексные (системные) испытания, и наконец приемочные испытания. Переход к следующему этапу возможен только после успешного завершения предыдущего этапа. Успешное завершение этапа означает отсутствие критических и значительных дефектов, выявленных в ходе испытаний. Дефекты с незначительным приоритетом могут быть устранены на следующих этапах, но должны быть задокументированы.

5.3.2. Порядок проведения предварительных (автономных) испытаний

Предварительные испытания проводятся в среде разработчика на основном тестовом стенде под управлением операционной системы Windows 11. Испытания выполняются разработчиком, ответственным за соответствующий модуль.

Сначала выполняются модульные тесты (JUnit 5) для всех компонентов backend-слоя. Запуск тестов производится командой `./gradlew test`. Все модульные тесты должны быть выполнены успешно: количество пройденных тестов должно составлять 100%, количество упавших тестов — 0. Результаты выполнения сохраняются в протокол.

Затем выполняются автономные тесты пользовательского интерфейса (Compose UI Test). Запуск UI-тестов производится аналогичной командой. Все UI-тесты должны быть выполнены успешно.

После автоматизированных тестов проводится ручная проверка каждого экрана и компонента. Проверяются: главный экран (отображение сводки, диаграммы, списка категорий); экран истории (отображение списка операций, работа фильтров); экран настроек (отображение списков категорий, работа кнопок добавления и удаления); диалог добавления и редактирования операций; диалог добавления категории; диалоги подтверждения удаления; работа навигационной панели; работа прогресс-бара «Гонка с призраком»; работа круговой диаграммы.

Выявленные на этапе предварительных испытаний дефекты фиксируются в протоколе испытаний. Дефекты с критическим и значительным приоритетом должны быть устранены до перехода к следующему этапу. Дефекты с незначительным приоритетом могут быть устранены в процессе комплексных испытаний.

5.3.3. Порядок проведения комплексных (системных) испытаний

Комплексные испытания проводятся на всех трех тестовых стендах (Windows 11, Ubuntu 22.04, macOS 13 при наличии). Испытания выполняются командой разработки под руководством Team Lead.

Испытания проводятся в соответствии с Чек-листом тестирования (Таблица 2 из Отчета по проекту, раздел 4.4). Выполняются тестовые сценарии ТС-01 — ТС-10, которые охватывают все функциональные и нефункциональные требования. Каждый тестовый сценарий выполняется последовательно на каждом тестовом стенде. Результаты выполнения каждого сценария фиксируются в протоколе с указанием: даты и времени

выполнения; тестового стенда (операционная система); результата (пройден / не пройден); описания выявленного дефекта (при не прохождении).

В ходе комплексных испытаний проводятся также специальные проверки нефункциональных требований.

Для проверки производительности фильтрации предварительно в базу данных загружается 1000 тестовых операций (500 доходов и 500 расходов) с помощью специального скрипта. Затем с помощью секундомера замеряется время отклика при переключении периодов на главном экране и при применении фильтров в истории. Измерения проводятся не менее пяти раз для каждого действия, фиксируется среднее значение. Результаты замеров заносятся в протокол.

Для проверки времени запуска приложения выполняется пять последовательных запусков приложения. Секундомер запускается в момент двойного клика по JAR-файлу и останавливается в момент полной отрисовки главного окна. Фиксируется среднее время запуска.

Для проверки безопасности от SQL-инъекций сначала проводится статический анализ кода: проверяются все классы в пакете `db.database_request` на наличие `PreparedStatement` с плейсхолдерами (?). Результаты статического анализа фиксируются в протоколе. Затем проводится динамический тест: в поле «Описание» при добавлении операции вводится строка `'; DROP TABLE Income_transactions; --`. После сохранения операции проверяется, что база данных не повреждена (таблицы на месте, данные не удалены).

Для проверки кроссплатформенности все тестовые сценарии TC-01 — TC-10 выполняются на каждом из трех тестовых стендов. Фиксируются любые расхождения в поведении приложения: отличия в отображении шрифтов, некорректная работа ввода с клавиатуры, различия в производительности.

Для проверки надежности имитируется сбой: в момент выполнения операции добавления записи приложение принудительно завершается через диспетчер задач (Windows) или команду `kill` (Linux). После повторного запуска проверяется целостность базы данных и сохранность ранее добавленных операций.

5.3.4. Порядок проведения приемочных испытаний

Приемочные испытания проводятся в присутствии заказчика (преподавателя) на основном тестовом стенде под управлением операционной системы Windows 11. Испытания проводит Team Lead, демонстрируя заказчику все ключевые функции приложения.

Демонстрация включает следующие этапы. Сначала демонстрируется запуск приложения и автоматическая инициализация базы данных (показывается, что при первом запуске создаются предустановленные категории). Затем демонстрируется добавление операции расхода с заполнением всех полей и добавление операции дохода. После этого демонстрируется редактирование созданной операции (изменение суммы и категории) и удаление операции (с подтверждением).

Далее демонстрируется экран истории: показывается полный список операций, затем демонстрируется фильтрация по типу (доходы, расходы, все) и фильтрация по временным периодам (день, неделя, месяц, год). Затем демонстрируется главный экран: показывается переключение между режимами «Доходы» и «Расходы», изменение круговой диаграммы, фильтрация данных по периодам, работа прогресс-бара «Гонка с призраком».

После этого демонстрируется экран настроек: показывается добавление новой категории, затем создание операции с использованием новой категории, и наконец попытка удаления использованной категории (с демонстрацией блокировки). Демонстрируется также удаление неиспользуемой категории (успешное).

Затем демонстрируется обработка ошибок: ввод отрицательной суммы, ввод даты, уходящей более чем на 1 день в будущее, попытка сохранения операции без выбора категории.

Заказчик имеет право задавать вопросы и запрашивать дополнительную демонстрацию любых функций приложения. По результатам демонстрации заказчик выносит вердикт о готовности продукта к внедрению, который фиксируется в протоколе испытаний.

5.3.5. Условия досрочного прекращения испытаний

Испытания могут быть досрочно прекращены в следующих случаях.

При обнаружении критического дефекта, блокирующего выполнение основных функций (невозможность добавления операции, потеря данных, невозможность запуска приложения), испытания приостанавливаются до устранения дефекта. После устранения испытания возобновляются с того этапа, на котором были прерваны.

При обнаружении несоответствия более чем 20% функциональных требований (из Раздела 3 настоящего документа) испытания прекращаются. Продукт отправляется на доработку. Назначается повторный цикл испытаний.

При отказе тестового оборудования или программного обеспечения, делающем невозможным продолжение испытаний, испытания переносятся на согласованный срок после восстановления работоспособности стенда.

5.4. Требования к персоналу, проводящему испытания

Испытания проводятся членами команды разработки проекта «Finlytics» под руководством руководителя проектной работы.

Разработчик, проводящий испытания, должен иметь опыт работы с операционными системами Windows, Linux и macOS на уровне уверенного пользователя; знать основы тестирования программного обеспечения (типы тестирования, структуру тестовых сценариев); уметь работать с системой контроля версий Git и средой разработки IntelliJ IDEA; понимать архитектуру разработанного приложения и его компоненты.

Team Lead, контролирующий проведение испытаний, дополнительно должен уметь анализировать выявленные дефекты, определять их приоритет и назначать ответственных за устранение; оформлять протокол испытаний и акт приемки; принимать решение о готовности продукта к переходу на следующий этап испытаний.

Заказчик (преподаватель), присутствующий на приемочных испытаниях, должен быть ознакомлен с Техническим заданием и Матрицей требований для объективной оценки соответствия продукта заявленным характеристикам.

6. Методы испытаний

6.1. Общие положения

В настоящем разделе приводится детальное описание методов проверки каждого требования, перечисленного в Разделе 3 настоящего документа. Для каждого требования указаны конкретные действия проверяющего, ожидаемый результат и ссылка на соответствующий тестовый сценарий (ТС) из Чек-листа тестирования.

Методы испытаний разделены на три группы в соответствии с типом требований: функциональные требования, нефункциональные требования и требования к документации.

Все испытания проводятся в порядке, установленном Разделом 5. Результаты каждой проверки фиксируются в Протоколе испытаний.

6.2. Методы испытаний функциональных требований

6.2.1. Метод проверки требования ФТ-01 (добавление операции)

Суть требования. Программа должна предоставлять пользователю возможность добавлять новые финансовые операции с указанием типа, суммы, категории, даты и описания.

Метод испытания. Функциональное тестирование с использованием позитивных тестовых данных. Проверка выполняется на основном тестовом стенде под управлением Windows 11.

Порядок проведения проверки. Запускается приложение. На нижней панели навигации нажимается кнопка «Добавить». В открывшемся диалоговом окне выбирается тип операции «Расход». В поле «Сумма (₽)» вводится значение «500». Поле «Дата» оставляется с текущей датой (или нажимается кнопка «Сегодня»). В списке категорий выбирается категория «Еда» (предполагается, что она существует по умолчанию). В поле «Описание» вводится текст «Обед». После заполнения всех полей нажимается кнопка «Создать операцию».

Ожидаемый результат. Диалоговое окно закрывается. На главном экране («Обзор») общая сумма расходов увеличивается на 500 рублей, баланс уменьшается на 500 рублей. При переходе на экран «История» в списке операций появляется новая запись с параметрами: тип — «Расход», сумма — «500», категория — «Еда», описание — «Обед», дата соответствует текущей. При повторной проверке через SQLite Browser в таблице expenses_transactions появляется новая запись с соответствующими значениями.

Ссылка на тестовый сценарий. ТС-01.

6.2.2. Метод проверки требования ФТ-02 (редактирование операции)

Суть требования. Программа должна предоставлять пользователю возможность редактировать существующие финансовые операции.

Метод испытания. Функциональное тестирование с использованием позитивных тестовых данных. Проверка выполняется на основном тестовом стенде.

Порядок проведения проверки. Предварительно в систему добавляется тестовая операция (например, «Обед» на 500 рублей в категории «Еда»). Затем открывается экран «История». Находится созданная операция и нажимается желтая кнопка с иконкой карандаша (редактирование). В открывшемся диалоговом окне сумма изменяется на «750», категория изменяется на «Транспорт». После внесения изменений нажимается кнопка «Сохранить изменения».

Ожидаемый результат. Диалоговое окно закрывается. В списке истории у отредактированной операции сумма меняется на 750 рублей, категория меняется на «Транспорт». На главном экране общая сумма расходов пересчитывается с учетом новой суммы. При повторной проверке через SQLite Browser в соответствующей таблице транзакций обновляются поля суммы и идентификатора категории.

Ссылка на тестовый сценарий. ТС-03.

6.2.3. Метод проверки требования ФТ-03 (удаление операции)

Суть требования. Программа должна предоставлять пользователю возможность удалять существующие финансовые операции, после чего операция перестает учитываться в статистике.

Метод испытания. Функциональное тестирование с использованием позитивных тестовых данных. Проверка выполняется на основном тестовом стенде.

Порядок проведения проверки. Предварительно в систему добавляется тестовая операция (например, «ТестУдаление» на 100 рублей). Открывается экран «История». Находится созданная операция и нажимается красная кнопка с иконкой корзины (удаление). В появившемся диалоговом окне подтверждения нажимается кнопка «Удалить».

Ожидаемый результат. Диалог подтверждения закрывается. Операция исчезает из списка истории. На главном экране общая сумма доходов или расходов пересчитывается без учета удаленной операции, баланс также пересчитывается. При повторной проверке через SQLite Browser запись в соответствующей таблице транзакций отсутствует.

Ссылка на тестовый сценарий. ТС-04.

6.2.4. Метод проверки требования ФТ-04 (предустановленные категории)

Суть требования. При первом запуске приложение должно автоматически создавать предустановленный набор категорий для доходов и расходов.

Метод испытания. Инспекция и функциональное тестирование с проверкой состояния базы данных после чистой установки.

Порядок проведения проверки. Приложение полностью закрывается. Файл базы данных Finlytics.db удаляется из директории приложения (если существует). Приложение запускается заново. После полной загрузки открывается экран «Настройки». Визуально проверяется содержимое левой панели («Категории доходов») и правой панели («Категории расходов»). Дополнительно через SQLite Browser проверяется содержимое таблиц Income_categories и expenses_categories.

Ожидаемый результат. В левой панели присутствуют категории доходов: «Зарплата», «Стипендия», «Фриланс», «Инвестиции», «Подарок». В правой панели присутствуют категории расходов: «Еда», «Транспорт», «Жилье», «Развлечения»,

«Образование», «Здоровье», «Одежда», «Коммунальные услуги». В таблицах базы данных содержатся соответствующие записи. При повторном запуске приложения дублирование категорий не происходит.

Ссылка на тестовый сценарий. Подготовка к ТС-01, ТС-06.

6.2.5. Метод проверки требования ФТ-05 (создание пользовательских категорий)

Суть требования. Пользователь должен иметь возможность создавать новые категории для доходов и расходов.

Метод испытания. Функциональное тестирование с использованием позитивных тестовых данных.

Порядок проведения проверки. Открывается экран «Настройки». В правой панели «Категории расходов» нажимается синяя кнопка с иконкой плюса («+»). В открывшемся диалоговом окне вводится название новой категории — «Кафе». Нажимается кнопка «Добавить». После закрытия диалога проверяется, что категория «Кафе» появилась в списке категорий расходов. Затем открывается диалог добавления новой операции (кнопка «Добавить» на нижней панели). В списке категорий проверяется наличие категории «Кафе» и возможность ее выбора.

Ожидаемый результат. Диалог добавления категории закрывается. Категория «Кафе» отображается в списке категорий расходов на экране «Настройки». При создании новой операции расхода категория «Кафе» доступна для выбора в соответствующем списке. При повторной проверке через SQLite Browser в таблице `expenses_categories` появляется новая запись с названием «Кафе».

Ссылка на тестовый сценарий. ТС-05.

6.2.6. Метод проверки требования ФТ-06 (удаление категорий с проверкой использования)

Суть требования. Система должна блокировать удаление категории, если она используется хотя бы в одной операции.

Метод испытания. Негативное функциональное тестирование.

Порядок проведения проверки. Предварительно создается новая категория, например «Кафе», и с использованием этой категории создается хотя бы одна операция расхода. Затем открывается экран «Настройки». В панели «Категории расходов» находится категория «Кафе» и нажимается красная кнопка с иконкой корзины (удаление). Анализируется содержимое появившегося диалогового окна и состояние кнопки «Удалить». Нажимается кнопка «Отмена». Дополнительно проверяется, что при попытке удалить категорию, не связанную с операциями (например, только что созданную, но еще не использованную), удаление выполняется успешно.

Ожидаемый результат. При попытке удалить категорию «Кафе», связанную с операцией, открывается диалоговое окно с предупреждением: «Существуют операции с этой категорией. Удаление категории невозможно, пока есть связанные операции». Кнопка «Удалить» в этом диалоге неактивна. При нажатии «Отмена» диалог закрывается, категория остается в списке. При попытке удалить неиспользуемую категорию диалог

предупреждения не отображается (или кнопка «Удалить» активна), и категория успешно удаляется.

Ссылка на тестовый сценарий. ТС-06.

6.2.7. Метод проверки требования ФТ-07 (финансовая сводка)

Суть требования. На главном экране должна отображаться финансовая сводка (баланс, доходы, расходы) за выбранный период.

Метод испытания. Функциональное тестирование с проверкой пересчета статистики при изменении периода.

Порядок проведения проверки. Предварительно в систему добавляются операции за разные даты: несколько операций за Март 2024 года (например, доход 10000 рублей, расход 3000 рублей) и несколько операций за Апрель 2024 года (доход 5000 рублей, расход 2000 рублей). Открывается экран «Обзор». В блоке «Период времени» выбирается «Месяц». С помощью стрелок навигации выбирается месяц «Март 2024». Фиксируются значения в блоках «Доходы», «Расходы» и «Баланс». Затем стрелками навигации выбирается месяц «Апрель 2024». Снова фиксируются значения. Проверяется, что доходы, расходы и баланс соответствуют суммам операций за выбранный месяц.

Ожидаемый результат. При выборе Марта 2024 доходы составляют 10000 рублей, расходы — 3000 рублей, баланс — 7000 рублей. При выборе Апреля 2024 доходы составляют 5000 рублей, расходы — 2000 рублей, баланс — 3000 рублей. При выборе периода «Всё время» доходы составляют 15000 рублей, расходы — 5000 рублей, баланс — 10000 рублей.

Ссылка на тестовый сценарий. ТС-07.

6.2.8. Метод проверки требования ФТ-08 (визуализация данных)

Суть требования. На главном экране должна отображаться круговая диаграмма, показывающая распределение доходов или расходов по категориям.

Метод испытания. Функциональное тестирование с визуальным контролем.

Порядок проведения проверки. Предварительно в систему добавляются операции доходов по разным категориям (например, «Зарплата» — 50000 рублей, «Фриланс» — 20000 рублей) и операции расходов по разным категориям (например, «Еда» — 10000 рублей, «Транспорт» — 5000 рублей, «Развлечения» — 3000 рублей). Открывается экран «Обзор». Нажимается кнопка «Расходы» (красная). Визуально оценивается круговая диаграмма: должны быть видны сегменты разных цветов, пропорциональные суммам расходов по категориям. Нажимается кнопка «Доходы» (зеленая). Визуально оценивается круговая диаграмма: сегменты должны соответствовать доходам по категориям.

Ожидаемый результат. В режиме «Расходы» на диаграмме отображаются сегменты, размер которых пропорционален суммам расходов: самый большой сегмент соответствует категории «Еда» (10000 рублей), средний — «Транспорт» (5000 рублей), самый маленький — «Развлечения» (3000 рублей). В режиме «Доходы» самый большой сегмент соответствует «Зарплате» (50000 рублей), меньший — «Фрилансу» (20000 рублей). При наведении на сегменты (при наличии соответствующей функциональности) или при просмотре списка категорий справа отображаются названия категорий, суммы и проценты.

Ссылка на тестовый сценарий. ТС-08.

6.2.9. Метод проверки требования ФТ-09 (фильтрация истории)

Суть требования. На экране истории должна быть возможность фильтровать список операций по типу и временному периоду.

Метод испытания. Функциональное тестирование с проверкой обновления списка при изменении фильтров.

Порядок проведения проверки. Предварительно в систему добавляются операции разных типов за разные даты: операции доходов за разные дни недели, операции расходов за разные дни недели. Открывается экран «История». Выбирается фильтр по типу «Доходы» — проверяется, что в списке отображаются только операции с типом «Доход». Выбирается фильтр по типу «Расходы» — проверяется, что в списке отображаются только операции с типом «Расход». Выбирается фильтр по типу «Все» — проверяется, что в списке отображаются все операции. Затем выбирается период «Неделя» и с помощью стрелок навигации выбирается конкретная неделя. Проверяется, что в списке отображаются только операции за выбранную неделю. Проверяется комбинация фильтров: «Расходы» + «Неделя».

Ожидаемый результат. При выборе фильтра «Доходы» в списке отсутствуют операции с типом «Расход». При выборе фильтра «Расходы» в списке отсутствуют операции с типом «Доход». При выборе периода «Неделя» в списке отсутствуют операции за пределами выбранной недели. При комбинации фильтров отображаются только операции, удовлетворяющие обоим критериям (например, только расходы за выбранную неделю). Список обновляется мгновенно после каждого изменения фильтра.

Ссылка на тестовый сценарий. ТС-08 (аналогично для истории).

6.2.10. Метод проверки требования ФТ-10 (прогресс-бар «Гонка с призраком»)

Суть требования. При выборе периода, отличного от «Всё время», должен отображаться прогресс-бар, показывающий соотношение расходов текущего периода с предыдущим.

Метод испытания. Функциональное тестирование с визуальным контролем.

Порядок проведения проверки. Предварительно в систему добавляются операции расходов за текущий месяц на общую сумму 5000 рублей и за предыдущий месяц на общую сумму 10000 рублей. Открывается экран «Обзор». Выбирается период «Месяц». Убедиться, что на экране присутствует компонент «Гонка с призраком» (под блоками статистики). Проверяется заполнение прогресс-бара и его цвет. Проверяется текст под прогресс-баром. Затем изменяются данные (добавляются расходы за текущий месяц до 12000 рублей) и проверяется изменение прогресс-бара.

Ожидаемый результат. При расходе текущего месяца 5000 рублей и прошлого 10000 рублей прогресс-бар заполнен на 50%, цвет — зеленый (менее 80%). Отображается текст: «Потрачено: 5000 Р» и «Лимит прошлого периода: 10000 Р». После добавления расходов до 12000 рублей прогресс-бар заполняется на 100% (так как более 100% не отображается), цвет меняется на красный (более 100%). При выборе периода «Всё время» компонент «Гонка с призраком» не отображается.

Ссылка на тестовый сценарий. ТС-10.

6.3. Методы испытаний нефункциональных требований

6.3.1. Метод проверки требования НФТ-01 (производительность фильтрации)

Суть требования. Время отклика интерфейса на действия пользователя не должно превышать 500 миллисекунд.

Метод испытания. Инструментальные замеры с использованием секундомера и нагрузочного тестирования.

Порядок проведения проверки. Предварительно с помощью специального скрипта в базу данных загружается 1000 тестовых операций (500 доходов и 500 расходов с разными датами). Приложение запускается на стенде для тестирования производительности (Intel Core i3, 8 ГБ ОЗУ). Открывается экран «Обзор». С помощью секундомера замеряется время от момента клика по кнопке периода «Месяц» до момента полного обновления интерфейса (изменения цифр в блоках статистики и диаграммы). Измерение повторяется пять раз, фиксируется среднее значение. Аналогичные замеры проводятся для переключения между режимами «Доходы» и «Расходы», а также для применения фильтров на экране «История».

Ожидаемый результат. Среднее время отклика при переключении периода не превышает 500 миллисекунд. Среднее время отклика при переключении между доходами и расходами не превышает 300 миллисекунд. Среднее время применения фильтров в истории не превышает 400 миллисекунд. Все замеры фиксируются в Протоколе испытаний.

Ссылка на тестовый сценарий. Требование проверяется в рамках ТС-07, ТС-08.

6.3.2. Метод проверки требования НФТ-02 (время запуска приложения)

Суть требования. Время от запуска JAR-файла до полной загрузки главного окна не должно превышать 3 секунд.

Метод испытания. Инструментальные замеры с использованием секундомера.

Порядок проведения проверки. Приложение закрывается (если запущено). Секундомер запускается в момент двойного клика по файлу Finlytics.jar. Секундомер останавливается в момент полной отрисовки главного окна приложения (когда все элементы интерфейса стали видны, а данные загружены). Измерение повторяется пять раз подряд (с полным закрытием приложения между запусками). Вычисляется среднее арифметическое значение. Тест проводится на стенде для тестирования производительности и на основном стенде.

Ожидаемый результат. Среднее время запуска на основном стенде не превышает 2 секунд. Среднее время запуска на стенде для тестирования производительности не превышает 3 секунд. При наличии в базе данных 1000 операций время запуска увеличивается не более чем на 0.5 секунды по сравнению с пустой базой.

Ссылка на тестовый сценарий. Отдельная проверка, не привязанная к конкретному ТС.

6.3.3. Метод проверки требования НФТ-03 (безопасность данных от SQL-инъекций)

Суть требования. Приложение должно использовать параметризованные SQL-запросы для исключения возможности SQL-инъекций.

Метод испытания. Статический анализ кода и динамическое негативное тестирование.

Порядок проведения проверки. Сначала проводится статический анализ: в среде разработки открываются все классы из пакета db.database_request (AddCategory.kt, AddIncomeTransaction.kt, AddExpensesTransaction.kt, UpdateIncomeTransaction.kt, UpdateExpensesTransaction.kt, DeleteIncomeTransaction.kt, DeleteExpensesTransaction.kt). Проверяется, что во всех SQL-запросах используются PreparedStatement с плейсхолдерами (?) вместо конкатенации строк. Затем проводится динамическое тестирование: открывается диалог добавления операции. В поле «Описание» вводится строка: `' ; DROP TABLE Income_transactions; --`. Остальные поля заполняются корректно. Операция сохраняется. После сохранения проверяется, что база данных не повреждена: приложение продолжает работать корректно, все операции отображаются, таблицы Income_transactions и expenses_transactions существуют и содержат все ранее добавленные записи.

Ожидаемый результат. При статическом анализе во всех классах обнаружено использование PreparedStatement. Ни в одном классе не обнаружено формирование SQL-запросов через конкатенацию строк с пользовательскими данными. При динамическом тестировании операция сохраняется как обычная запись (введенная строка попадает в поле описания без выполнения как SQL-команды). База данных не повреждена, приложение не «падает». При просмотре через SQLite Browser в поле описания добавленной операции отображается введенная строка `' ; DROP TABLE Income_transactions; --` как обычный текст.

Ссылка на тестовый сценарий. Отдельная проверка безопасности.

6.3.4. Метод проверки требования НФТ-04 (кроссплатформенность)

Суть требования. Приложение должно корректно работать на Windows, macOS и Linux.

Метод испытания. Тестирование совместимости на разных операционных системах.

Порядок проведения проверки. Исполняемый JAR-файл копируется на каждый из трех тестовых стендов (Windows 11, Ubuntu 22.04, macOS 13 при наличии). На каждом стенде выполняется запуск приложения. Проверяется, что интерфейс отображается корректно (нет наложения элементов, все кнопки видны, текст читаем). На каждом стенде последовательно выполняются тестовые сценарии TC-01, TC-03, TC-04, TC-05, TC-06, TC-07, TC-08. Фиксируются любые различия в поведении: отличия в отображении шрифтов, некорректная работа ввода с клавиатуры, различия в производительности, сбои при работе с файловой системой.

Ожидаемый результат. На всех трех операционных системах приложение запускается без ошибок. Интерфейс отображается корректно, все элементы управления доступны. Все тестовые сценарии выполняются успешно. Возможны незначительные отличия в отображении шрифтов (зависит от наличия установленных шрифтов в ОС), но все тексты должны быть читаемы. Отличия в производительности допустимы, но не должны превышать 30% по сравнению с Windows (базовая платформа разработки). При выявлении критических различий (например, полная неработоспособность на одной из ОС) требование считается невыполненным.

Ссылка на тестовый сценарий. Проверка всех ТС на разных ОС.

6.3.5. Метод проверки требования НФТ-05 (целостность данных)

Суть требования. При некорректном завершении работы приложения база данных не должна повреждаться.

Метод испытания. Тестирование надежности с имитацией сбоев.

Порядок проведения проверки. В системе создается несколько операций (например, 5-10 записей). Затем, в момент выполнения операции добавления новой записи (после нажатия кнопки «Создать операцию», но до момента завершения записи в БД), приложение принудительно завершается через диспетчер задач (Windows) или команду kill (Linux). После этого приложение запускается снова. Проверяется, что все операции, успешно сохраненные до момента сбоя, остались в базе данных. Проверяется, что база данных открывается без ошибок (отсутствует сообщение о повреждении файла Finlytics.db). Проверяется, что приложение запускается и работает корректно.

Ожидаемый результат. После принудительного завершения приложения и повторного запуска все ранее созданные операции (до момента сбоя) присутствуют в базе данных. Операция, добавление которой не было завершено, может отсутствовать (это допустимо), но не должна быть записана частично. Файл базы данных не поврежден, SQLite Browser открывает его без ошибок. Приложение запускается и работает корректно, новые операции добавляются без проблем.

Ссылка на тестовый сценарий. Отдельная проверка надежности.

6.3.6. Метод проверки требования НФТ-06 (удобство использования)

Суть требования. Все элементы управления должны иметь всплывающие подсказки, а сообщения об ошибках должны быть понятными.

Метод испытания. Визуальный контроль и функциональное тестирование.

Порядок проведения проверки. Последовательно наводятся курсор мыши на все интерактивные элементы интерфейса: кнопки навигации («Обзор», «История», «Настройки», «Добавить»), кнопки фильтров («Доходы», «Расходы», «Все»), кнопки периодов («День», «Неделя», «Месяц», «Год», «Всё время»), кнопки навигации по дате (стрелки «влево» и «вправо»), кнопки добавления категорий («+»), кнопки редактирования (карандаш) и удаления (корзина). Проверяется, что при наведении появляется всплывающая подсказка с пояснением назначения элемента. Затем проводятся проверки сообщений об ошибках: в диалоге добавления операции вводится отрицательная сумма — проверяется текст ошибки; вводится некорректная дата — проверяется текст ошибки; предпринимается попытка удалить используемую категорию — проверяется текст предупреждения.

Ожидаемый результат. Все кнопки и поля ввода имеют всплывающие подсказки (например, при наведении на кнопку «Добавить» появляется подсказка «Создать новую операцию»). Сообщения об ошибках формулируются на русском языке, понятны пользователю без специальных знаний: «Введите сумму > 0», «Дата не может быть более чем на 1 день в будущем», «Существуют операции с этой категорией. Удаление категории невозможно, пока есть связанные операции». Сообщения об ошибках выделены цветом (красным) и расположены рядом с проблемным полем ввода.

Ссылка на тестовый сценарий. ТС-02, ТС-06, ТС-09.

6.4. Методы испытаний требований к документации

6.4.1. Метод проверки требования ДТ-01 (документация разработчика)

Суть требования. Исходный код должен быть документирован в соответствии со стандартом KDoc.

Метод испытания. Инспекция исходного кода и проверка сгенерированной документации.

Порядок проведения проверки. В среде разработки IntelliJ IDEA открывается проект Finlytics. Выполняется команда `./gradlew dokkaHtml` для генерации документации в формате HTML. Сгенерированная документация открывается в веб-браузере. Проверяется, что для всех публичных классов (например, `FinanceViewModel`, `FinanceRepository`, `Operation`, `AddCategory`, `GetIncomeTransactions`) присутствуют KDoc-комментарии с описанием назначения. Проверяется, что для всех публичных функций присутствуют комментарии с описанием параметров (тег `@param`) и возвращаемых значений (тег `@return`), а для функций, выбрасывающих исключения — тег `@throws`. Проверяется отсутствие публичных элементов без документации.

Ожидаемый результат. Документация успешно генерируется без ошибок. В сгенерированном HTML-файле присутствуют страницы для всех основных классов и функций. Каждый публичный класс имеет описание. Каждая публичная функция имеет описание параметров и возвращаемого значения. Отсутствуют публичные элементы, помеченные как «недокументированные».

Ссылка на тестовый сценарий. Отдельная проверка документации.

6.4.2. Метод проверки требования ДТ-02 (документация пользователя)

Суть требования. Программа должна сопровождаться документацией пользователя в виде GitHub Wiki.

Метод испытания. Инспекция Wiki-страниц и проверка полноты описания.

Порядок проведения проверки. Открывается браузер и переходим по адресу: <https://github.com/Hohrandrey/Finlytics/wiki>. Проверяется наличие всех обязательных разделов: введение (назначение и область применения); требования к аппаратному и программному обеспечению; инструкция по установке и запуску; описание пользовательского интерфейса (все экраны); инструкции по выполнению основных операций (добавление, редактирование, удаление операций, управление категориями, фильтрация); описание функционала «Гонка с призрак»; ответы на часто задаваемые вопросы. Проверяется, что описание соответствует актуальной версии приложения (версия 1.0.0). Проверяется наличие скриншотов экранов приложения (или ссылок на них).

Ожидаемый результат. Wiki содержит все перечисленные разделы. Инструкции по установке и запуску полны и позволяют пользователю, не знакомому с проектом, успешно запустить приложение. Описание интерфейса соответствует реальному виду приложения. Инструкции по выполнению операций корректны и позволяют повторить описанные действия. Скриншоты (при наличии) соответствуют текущей версии интерфейса.

Ссылка на тестовый сценарий. Отдельная проверка документации.